WEALTH PULSE: TECHNOLOGY ARCHITECTURE DOCUMENT

Deliverable ID: WP-ARCH-02-0001

Project Title: Wealth Pulse Cross-Asset Portfolio and Educational Suite

Date: June 17, 2026

Table of Contents

DOCUMENT METADATA

Document Ownership

Name	Position / Role	Organization	Email
Gage Radtke	Lead Software Engineer / Project Manager	Wilmington University	gradtke001@wilmu.edu
Dr. Charles Bachman	Capstone Advisor / Project Reviewer	Wilmington University	cbachman@wilmu.edu

Revision History

Date	Revision	Summary of Changes	Author
05/17/2026	0.1	Initial Conceptual Draft and Project Initialization	G. Radtke
06/04/2026	1.0	Phase II Iteration: Analysis and Business Requirements Locking	G. Radtke
06/17/2026	2.0	Scope Optimization: Deferring AI Roadmap; Structuring Architecture via CSS Modules, Component-Interface Definitions, and Caching Controls	G. Radtke

1. ARCHITECTURE OVERVIEW

1.1 Architecture Scope

This document establishes the foundational software, hardware, security, and data architecture for *Wealth Pulse*. The system is specifically engineered to bridge the tracking gap between traditional equities and physical alternative commodities (specifically precious metal stockpiles monitored by raw weight parameters in ounces and spot valuation metrics).

Per an administrative schedule optimization, all artificial intelligence (AI) fiduciary prompt pipelines have been successfully deferred to a future engineering roadmap. The architectural scope of this Minimum Viable Product (MVP) is explicitly constrained to delivering a highly stable, hyper-secure Asset Manager Ledger Tool and an interactive, client-side Financial Literacy Learning Center.

The architecture defines the components, interfaces, and behaviors of the system. It governs how financial data is compiled locally, transmitted across network infrastructure, validated against security frameworks, and rendered deterministically to the user.

1.2 Architecture Domains Summary

The development, validation, and execution boundaries of this project impact the following information systems and structural domains:

Technical Domain	Status	Impact Description
Data Architecture & Modeling	Impacted	Relational data mapping inside a local PostgreSQL instance.
Distributed Application Services	Impacted	Multi-layered Spring Boot API coordinating structural logic.
Client UI Presentation Layer	Impacted	Single Page Application decoupled into React Feature Modules.
Integration Architecture	Impacted	Secure outbound HTTPS connections to Alpha Vantage and GoldAPI.io.
Security Infrastructure	Impacted	Stateless JWT verification and one-way cryptographic hashing blocks.

1.3 Architecture Goals and Constraints

1.3.1 Availability

Because *Wealth Pulse* serves as a personal asset evaluation tool executing within an individual deployment footprint rather than a commercial high-frequency banking enterprise, the system is classified under an Urgent/Standard Development Recovery classification. The runtime engine is configured to support immediate, continuous localized failover on the host architecture. System

availability is targeted at 99.5% during active development intervals, excluding scheduled system modifications executing on the local platform daemon.

1.3.2 Scalability

The initial operational capacity of the local system is engineered to handle an isolated solo-user environment efficiently. However, the system architecture utilizes a stateless application layer design pattern. By decoupling the presentation view from database row states, the underlying components can easily be moved to cloud infrastructures (e.g., AWS or Heroku clusters) to scale seamlessly from a single user up to thousands of concurrent accounts without requiring code refactoring.

1.3.3 Extensibility

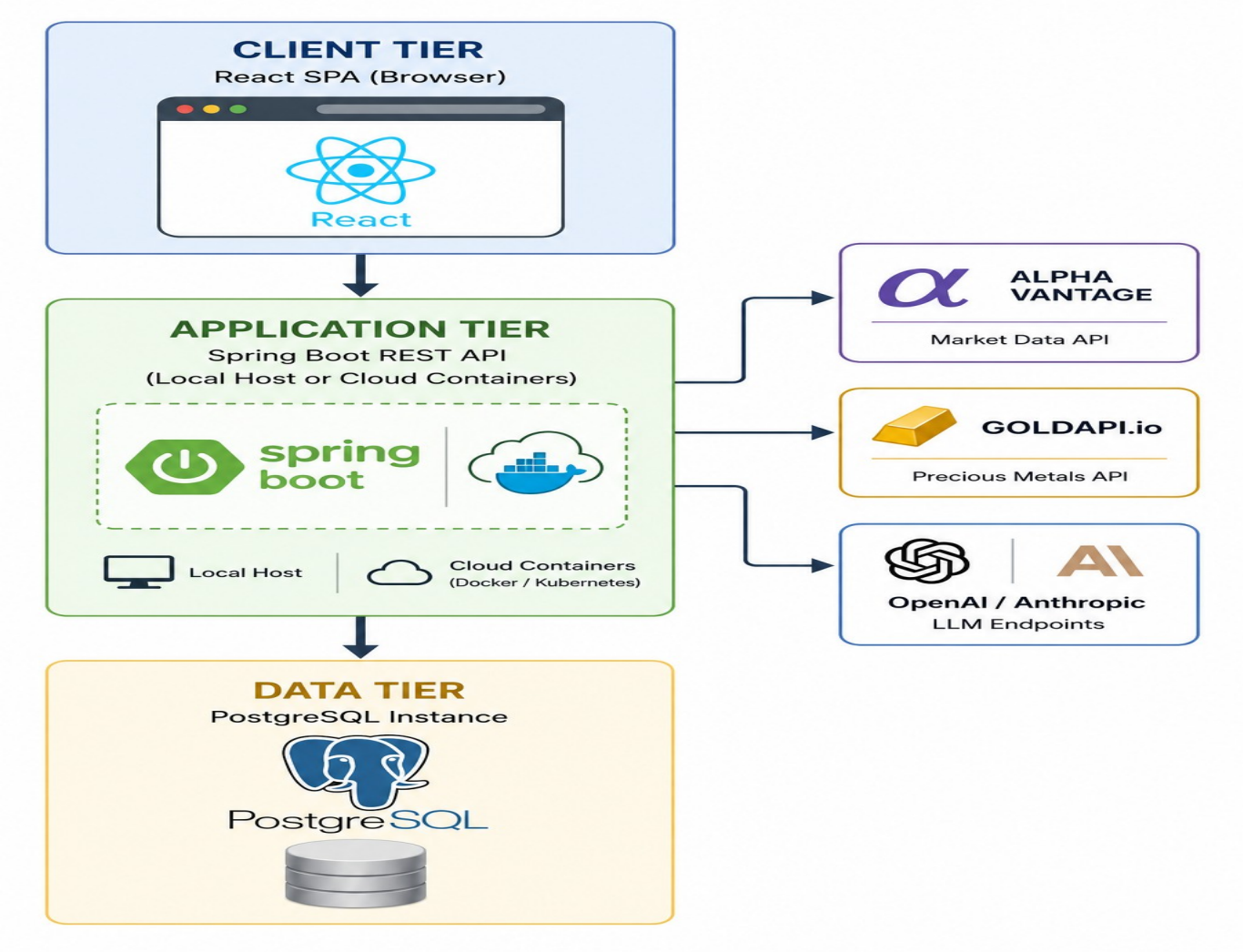
Extensibility is a primary structural goal of this architecture, particularly given the deferred status of the AI components. By utilizing standard Spring Data JPA data definitions and building clean REST controllers on the backend, the system ensures that future components—such as automated portfolio risk assessment bots or large language model (LLM) advisory interfaces—can quickly hook into existing asset endpoints without destabilizing the core codebase.

1.3.4 Re-usability

Code re usability is institutionalized by using custom utility calculation hooks and shared presentation wrappers. Structural operations like currency conversions, date manipulation formatting, and asset weight validations are isolated into reusable utility functions across both frontend and backend codebases, reducing duplicate code and minimizing maintenance overhead.

1.4 System Context Model

The System Context Model defines how the structural boundaries of the *Wealth Pulse* ecosystem connect out to adjacent environments and external actors.



1.5 Process Model Overview

The user journey flows through two core operational pathways:

1. **Asset Management Lifecycle:** The user authenticates via the login card, views their current net worth on the dashboard, navigates to the asset ledger, inputs a holding position (e.g., Gold bullion, declared in ounces), and triggers an immediate recalculation of their financial portfolio layout.
2. **Financial Literacy Lifecycle:** The user accesses the isolated Learning Center page, where interactive, client-side data modules parse financial concepts (such as equity options pricing or diversification indexing) without making state modifications to the primary financial database.

2. LOGICAL ARCHITECTURE

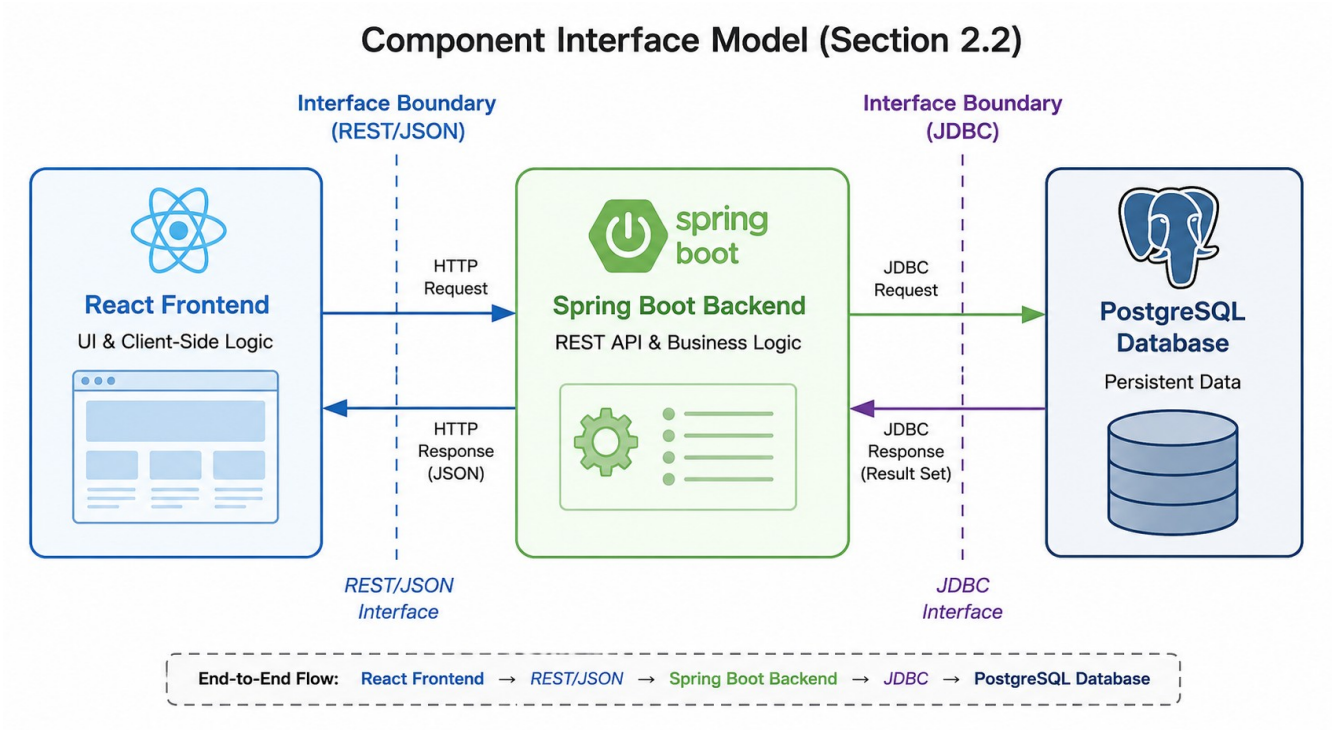
2.1 "As-Is" Model

Traditional consumer personal finance tools focus almost exclusively on regular stock brokerage interfaces, forcing users tracking alternative assets like physical precious metals to perform manual spreadsheet calculations to determine real-world melt value against live spot prices. The current "As-Is" environment is completely fragmented, decentralized, and lacks a uniform, secure, and combined data processing point.

2.2 "To-Be" Conceptual Model

The "To-Be" model unifies these disparate asset classes into a single secure platform. To achieve this, the architecture defines the components, interfaces, and behaviors of the system.

- **Components as Building Blocks:** The components are the building blocks for the system. These components may be built from scratch (such as our custom React ledger cards and the Spring Boot valuation math services) or re-used from an existing component library (such as standard Axios client packages and Spring Security filters). The components refine and capture the meaning of details from the requirement document, transforming abstract user requirements into tangible, executable blocks of software.
- **Component Composition and Interfaces:** The components are composed with other components using their interfaces. An interface forms a common boundary of two components. The interface is the architectural surface where independent components meet and communicate with each other. Over the interface, components interact and affect each other. In *Wealth Pulse*, this is primarily demonstrated by the REST API boundary, where the independent UI layer meets the application logic layer via structured JSON endpoints.
- **Interface-Driven Behavior:** The interface defines a behavior where one component responds to the stimuli of another component's actions. For instance, when a user enters an asset transaction and clicks "Submit," this client-side stimulus triggers an immediate corresponding behavior in the backend validation component, causing it to update the database state and return a refreshed portfolio calculation to the view.



2.3 Logical Models

The architecture is logically distributed across three independent layers, establishing clean separation of concerns:

1. **Presentation Layer (React UI):** Composed of feature-driven sub-directories (auth, assets, dashboard, learning). This layer handles user interaction and processes visual elements.
2. **Business Logic Layer (Spring Boot):** Intercepts incoming network operations, executes math verification parameters, validates data structures via Data Transfer Objects (DTOs), and coordinates external asset market validations.
3. **Data Persistence Layer (PostgreSQL):** Coordinates relational database operations, locking down user transaction tables, account profiles, and rate-limiting cache fields.

3. HARDWARE ARCHITECTURE

3.1 Tiers

The physical tier footprint is optimized for a highly efficient local deployment model using a decentralized **3-Tier Hardware Architecture**:

- **Client Tier:** Standard modern desktop browser environment interpreting local single-page React assets.

- **Application Server Tier:** Local Java virtual machine sandbox parsing compiled bytecode via Spring Boot.
- **Database Tier:** Persistent PostgreSQL data storage daemon managing active local disk write operations.

3.2 Client Hardware Models

- **Development Hardware Profile:** HP Laptop running an advanced multi-core AMD Ryzen processing engine.
- **Development Operating System:** Arch Linux (Rolling Release kernel configuration).
- **Development Environment IDE:** Code-OSS (Open-Source VS Code distribution).
- **Production Deployment Footprint:** Optimized to target standard modern client hardware runtimes executing Chromium browser architecture.

3.3 Server Hardware Models

During the MVP implementation phase, the server hardware environment is mapped locally to the host development machine, running via standard loopback network configurations (localhost) to provide rapid execution loops. The configuration is prepared to easily shift to cloud-allocated compute instances running an OpenJDK runtime container.

3.4 Mass Storage Devices

Data persistence is handled via the machine's local solid-state storage array, structured through the PostgreSQL engine. Relational table structures are modified using standard WAL (Write-Ahead Logging) protocols to guarantee complete transaction safety and prevent loss during local system interruptions.

4. NETWORK ARCHITECTURE

4.1 Overview

The network topology routes data traffic securely between presentation, business logic, and external data services. During local development, communication relies on internal system ports. This layout scales cleanly to public networks using industry-standard transport security measures.

4.2 Technology

- **Local Presentation Pipeline:** Port 5173 (Vite local loopback server compilation).

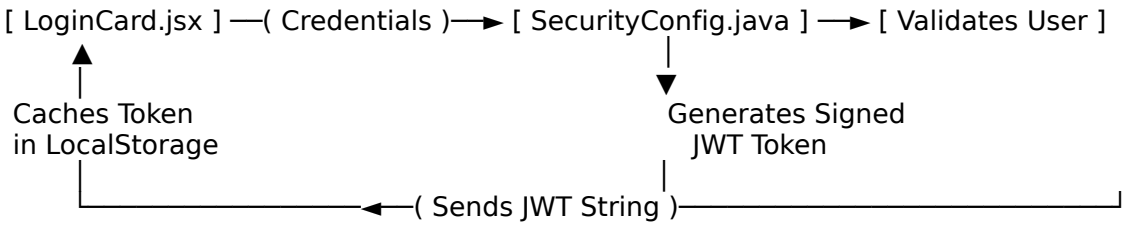
- **Local Application API Pipeline:** Port 8080 (Embedded Apache Tomcat server execution).
- **Database Connection Channel:** Port 5432 (PostgreSQL local engine access).
- **Network Protocol Standards:** All inter-process communication between browser and server components maps to standard HTTP/1.1 REST protocols. When deployed to a public staging server, the system enforces TLS 1.3 encryption across Port 443, dropping unencrypted port 80 traffic to block network packet-sniffing exploits.

5. SECURITY ARCHITECTURE

5.1 Security Overview

Because *Wealth Pulse* tracks highly sensitive user net-worth data and precious metal assets, the security framework implements strict access controls across all application layers. Security operations are divided into two main mechanisms:

1. **Cryptographic Data Protection at Rest:** User password strings are processed using a **BCrypt Password Encoder** within the Spring Security module. BCrypt applies an automatically shifting computational salt to each entry, generating an irreversible hash. This approach ensures that even if the underlying database is compromised, user credentials remain secure against rainbow-table attacks.
2. **Stateless Authentication in Flight (JWT Framework):** Authentication is completely decentralized using **JSON Web Tokens (JWT)**. This process eliminates the need for high server-side session memory storage:



Every subsequent client request triggers the **Axios Interceptor** inside api/interceptors.js. This automatically injects the security credential header string (Authorization: Bearer <JWT_TOKEN>) into the outbound request. The Spring backend verifies the token's cryptographic signature before processing any data queries.

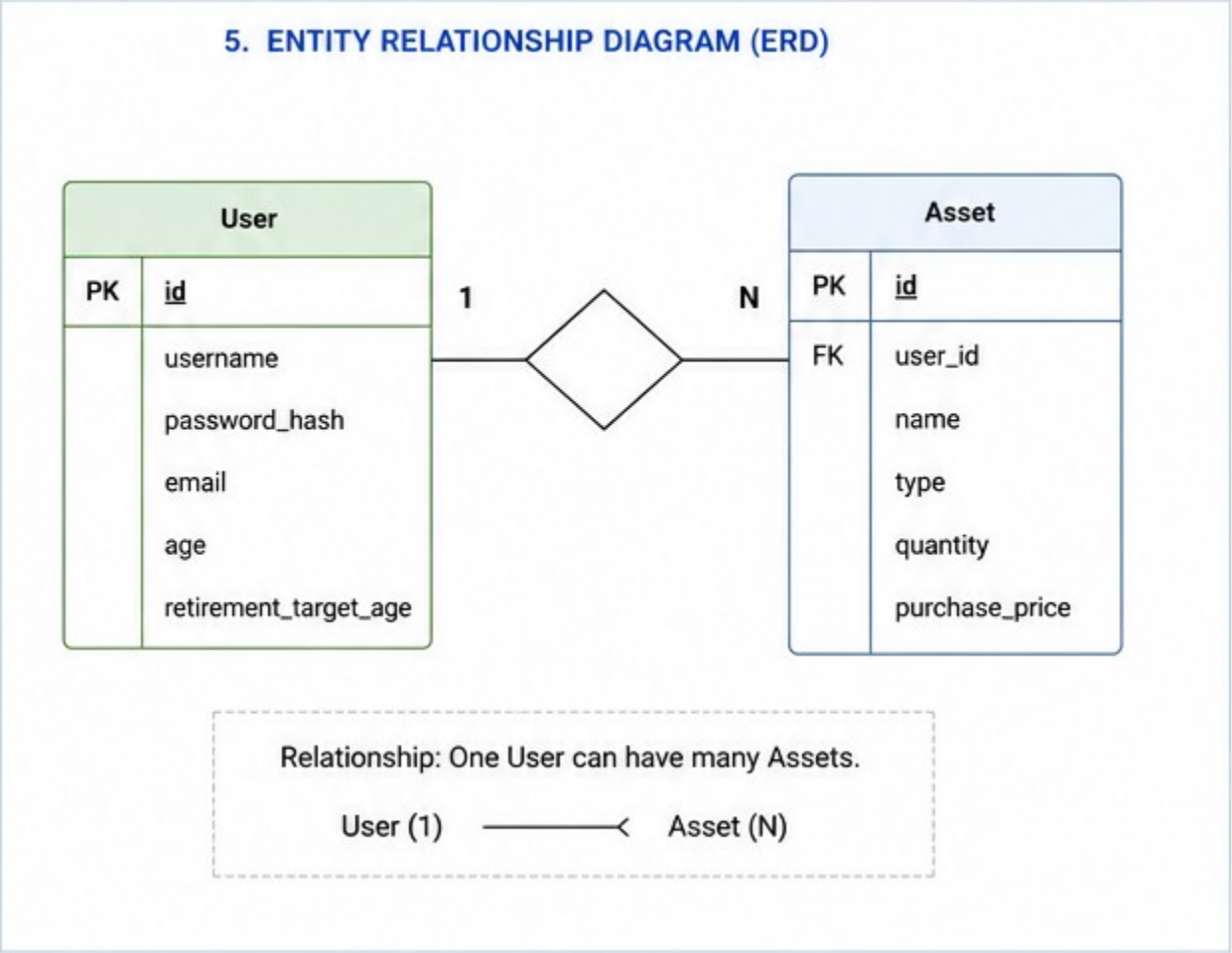
6. DATA ARCHITECTURE

6.1 Data Architecture Summary

The application data structure is explicitly relational, managed via an object-relational model using Hibernate and Spring Data JPA. The storage setup handles three distinct logical datasets: user profile records, asset positions, and cached market valuation metrics.

6.2 Conceptual Data Model

The conceptual data model defines the core system entity records, primary key constraints, and relational associations:



6.3 Database Sizing

For the localized solo-user capstone MVP, database sizing demands are minimal. The storage footprint requires less than 50MB of local disk space. To support future user scalability, all numeric data definitions for asset quantity allocations are mapped to high-precision DECIMAL(19,4) data types rather than floats, ensuring exact mathematical precision down to fractional asset ounces.

6.4 Logical Data Model

Data records map directly to physical database schemas via the system model components. To ensure clear separation between layers, database configurations use **Data Transfer Objects (DTOs)**. The application controllers process incoming data payloads as AssetDto objects, which the AssetMapper.java service converts into structural database entities before data persistence. This prevents internal database layouts from being exposed over the public network.

7. DATA INTEGRATION ARCHITECTURE

7.1 Integration Catalog

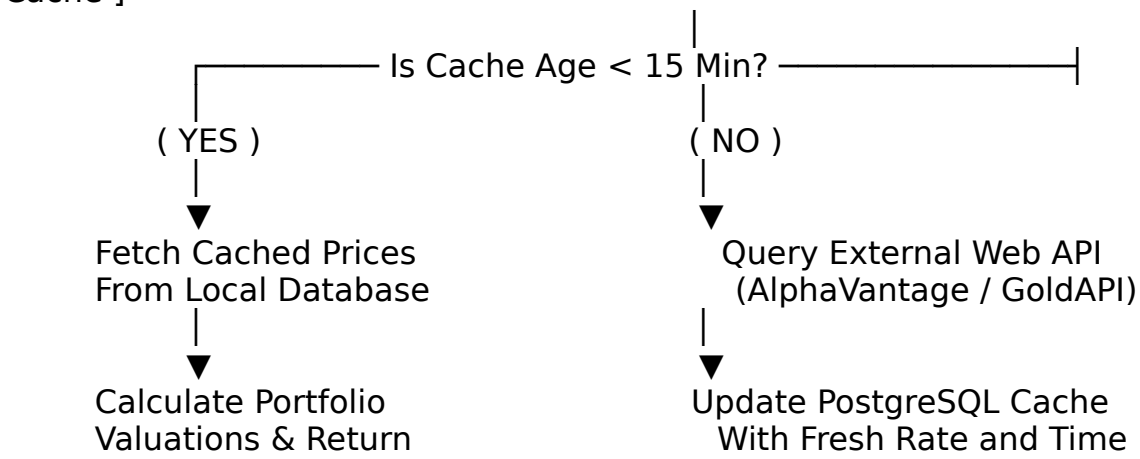
To provide accurate, real-time portfolio tracking, the backend service layer integrates with two external financial data providers via outbound REST operations:

External API Target	Data Category	Protocol Type	Authentication Mechanism
Alpha Vantage API	Traditional Equities Quotes	HTTPS GET (JSON)	Private Query Parameter API Token
GoldAPI.io Service	Precious Metals Spot Prices	HTTPS GET (JSON)	Private Secure Header Request Key

7.2 Data Attribute Mapping & Caching Mechanisms

Free-tier financial APIs operate under strict rate limits (such as a maximum of 5 requests per minute). If a user rapidly refreshes their frontend dashboard, these limits can easily be exceeded, resulting in system errors.

To mitigate this constraint, the system integrates a **Database-Backed Caching Strategy** directly into the AssetService.java data component loop:



This caching layer isolates the external integration boundaries, ensuring consistent app performance and complete platform stability under free-tier constraints.

8. USER INTERFACE ARCHITECTURE

8.1 User Interface Summary

The user interface is engineered as a decoupled React.js Single Page Application, running on the Vite compilation suite. To resolve styling conflicts where global CSS rules overwrite each other, the UI layer uses **Component-Specific CSS Modules**.

By appending the .module.css naming suffix to localized style definitions (such as Assets.module.css), the compilation compiler automatically hashes style classes at build time. A class rule defined as .card is rewritten into a unique runtime string (e.g., ._card_a1b2c_3), ensuring total design isolation across distinct app screens.

8.2 Navigation

The application routing engine is handled client-side via routes.jsx, providing quick view switches without page reloads. Navigational pathways are cleanly divided into three distinct workspace interfaces:

- /auth: Coordinates public access for security credential input (LoginCard.jsx, RegisterCard.jsx).
- /dashboard: Displays financial status readouts, asset distributions, and portfolio metrics.
- /ledger: Enforces CRUD capabilities for asset records via modal input fields and datatables.
- /learning: Hosts the isolated education engine, parsing investment concepts via client-side calculations.

8.3 User Interface Architecture Layouts

The app screen layout utilizes a structured design approach through DashboardLayout.jsx. The screen layout is divided into three functional areas:

1. **Navigation Core (DashboardSidebar.jsx):** A fixed vertical bar controlling page routing context.
2. **Context Header (DashboardHeader.jsx):** A top status bar tracking user account information and session controls.

3. **Dynamic Viewport Container:** A responsive content area that updates to render the selected feature view (DashboardPage.jsx, AssetManager.jsx, or LearningCenter.jsx) based on active user actions.

Technology Architecture Document — Current Implementation Addendum

Deliverable: WP-ARCH-02-0001 **Revision:** 3.0 **Date:** August 3, 2026 **Author:** Gage Radtke

Revision authority: This addendum records the implemented repository state. Where it conflicts with the earlier design-phase pages, this addendum governs.

Implemented System

Layer	Current implementation
Client	React 19 and Vite 8 single-page application. Source is organized by feature; styling uses shared and feature CSS files, not CSS Modules.
API	Java 21, Spring Boot 4.1, Spring MVC, Spring Security, Bean Validation, and stateless JWT authentication. Local port: **8283**.
Persistence	One PostgreSQL database named **wealth_pulse**, accessed through Spring Data JPA. User-owned assets are filtered by owner ID.
Integrations	Alpha Vantage for stock quotes/history, GoldAPI for metals, RSS feeds for news, and public Yahoo quote pages for market indices. Provider failures are isolated from supported cached behavior.

Request and Component Flow

```
React component → feature hook → feature API module → shared Axios client
→ JWT filter → controller → business service → repository → wealth_pulse

Pricing: AssetPricingService → AssetService cache check
→ MarketCacheRepository → external provider only when refresh is needed
```

DTO mapping is implemented by **AssetRequestMapper** using **CreateAssetRequest** and **UpdateAssetQuantityRequest**. References to *AssetDto* or *AssetMapper.java* in the original document are obsolete.

Current Data Model

Table	Purpose
users	Credentials, email, username, and account creation time.

assets	Single-table inheritance for stocks, ETFs, bonds, and metals; owned through user_id.
asset_transactions	Buy, sell, opening balance, dividend, interest, and fee ledger records.
portfolio_snapshots	Daily total and asset-class values used for performance series.
market_cache / historical_cache	Current and historical provider data used to reduce network traffic.
profile_pictures	Size-limited user image storage.

Money-oriented ledger, cache, and snapshot fields use BigDecimal/numeric values. The inherited asset entity currently persists quantity, price, and amount paid as Double/double precision; therefore the earlier claim that every asset numeric field uses DECIMAL(19,4) is not accurate.

Performance Feature Decomposition

PortfolioPerformanceService coordinates the use case. **PortfolioSnapshotService** owns snapshots, **PortfolioMetricsService** owns ledger calculations, and **PortfolioReportService** builds benchmark, series, allocation, and performer output.

Deployment and Security Boundaries

- Frontend default: http://localhost:5173
- Backend default: http://localhost:8283
- Database default: jdbc:postgresql://localhost:5432/wealth_pulse
- JWT_SECRET must be supplied outside the local dev profile.
- CORS origins and API/database credentials are environment-configurable.
- No automatic local failover or 99.5% availability mechanism is implemented; those statements remain goals, not verified service levels.